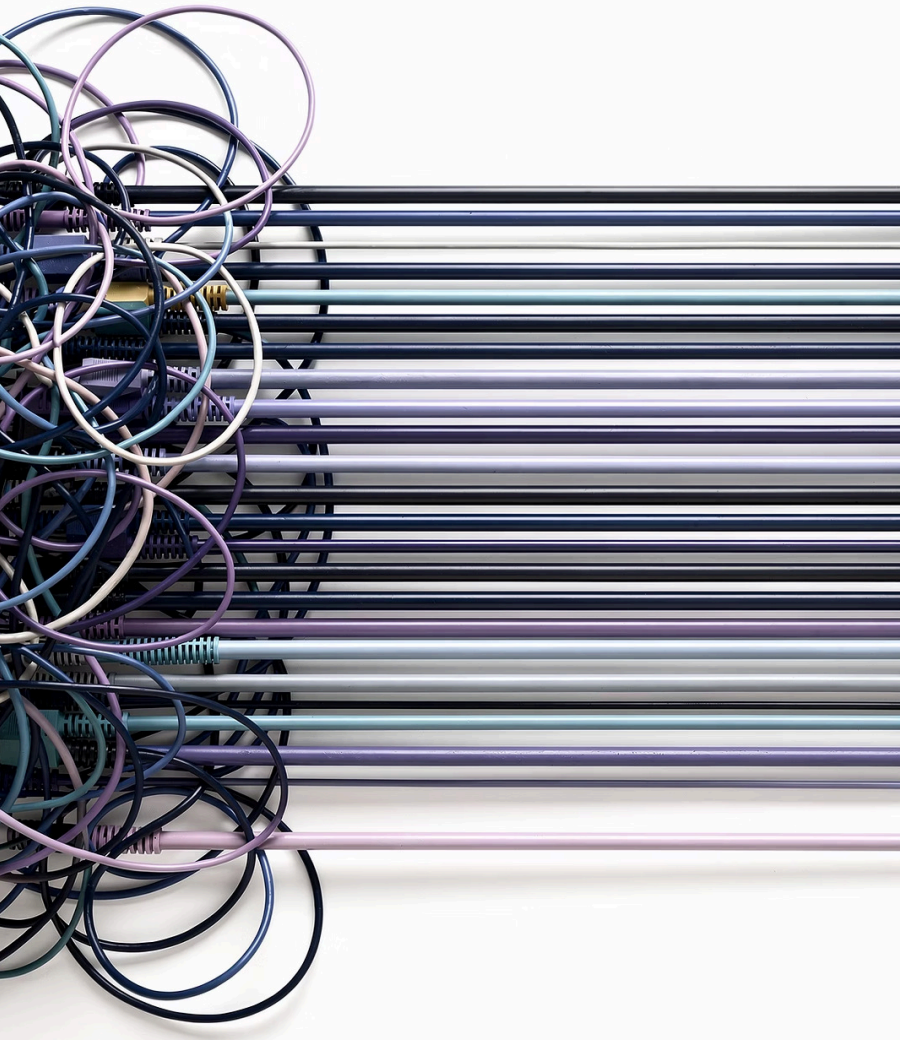


תל-חי
אוניברסיטת
קריית שמונה בגליל

Introduction to Data Science

Data Cleaning & Preprocessing

Dr. Marcelo Feighelstein



Why Data Cleaning Matters

🗑️ Garbage In

Bad data leads to bad models — no matter how powerful the algorithm.

📉 Model Failure

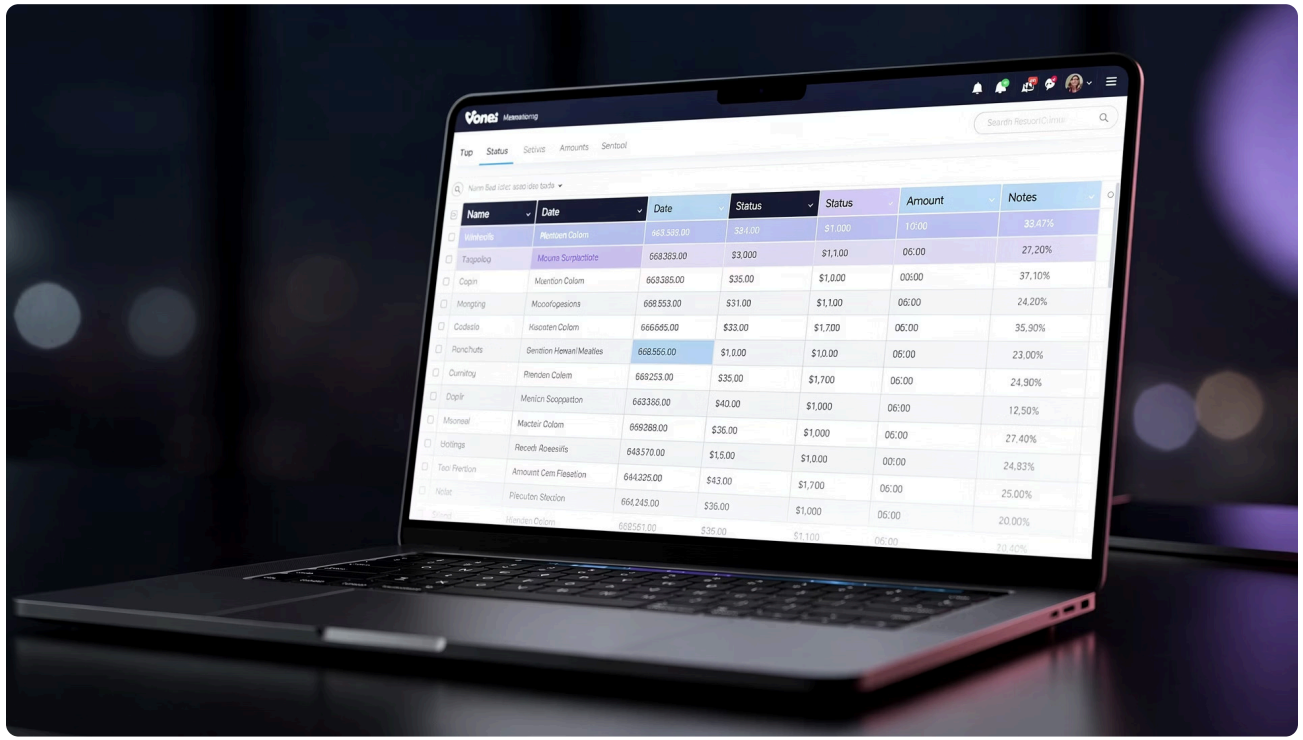
A model trained on dirty data predicts incorrectly, consistently.

🌍 Real Stakes

Medical diagnoses, loan approvals, and fraud detection all depend on clean data.

⚠️ Rule #1: Even the best ML model cannot fix bad input data.

What Is Data Cleaning?



Definition

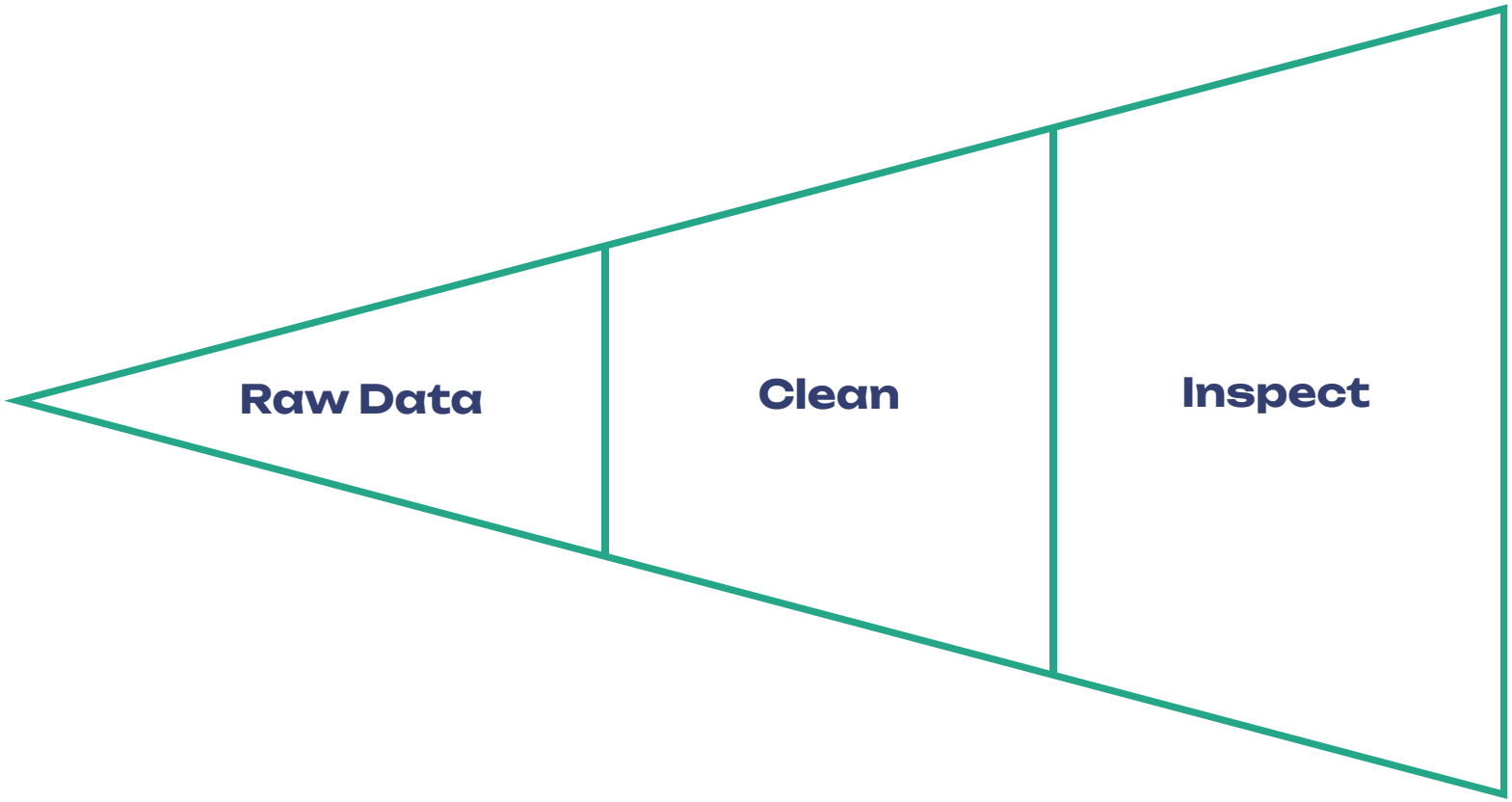
Data cleaning is the process of detecting and correcting errors, inconsistencies, and gaps in a dataset so it becomes **usable and reliable**.

Goal

- Make data accurate and consistent
- Ensure algorithms can process it
- Improve model performance and trust

Cleaning happens early in the ML pipeline — and often repeats.

The Data Cleaning Pipeline



This pipeline is **iterative** — you rarely clean data perfectly in one pass. Expect to loop back as you discover new issues at each stage.

Types of Data Problems



Irrelevant Features

Columns that add noise, not signal.



Missing Values

Empty cells where data should exist.



Wrong Formats

Dates as text, numbers as strings, mixed types.



Outliers

Extreme values that distort analysis.

Removing Unnecessary Data

Not all columns — or rows — belong in your model. Keeping them adds noise and slows training.

❌ Before Cleaning

ID	Name	Age	Salary
001	Alice	29	50000
002	Bob	34	62000
002	Bob	34	62000
003	Carol	41	71000

✅ After Cleaning

Age	Salary
29	50000
34	62000
41	71000

ID and Name removed (irrelevant). Duplicate row dropped.



Challenge Question 🤔

> How do we know which columns are unnecessary?

Domain Knowledge

Does this feature logically affect the outcome we're predicting?

Correlation Analysis

Does this column have any statistical relationship with the target?

Gut Check

Would a human expert use this information to make the same decision?

Missing Values — Why They Matter



Three Core Problems

→ Break Algorithms

Most ML models cannot handle `NaN` values — they crash or error out.

→ Bias Results

If data is missing non-randomly, analysis skews toward the rows that *do* have values.

→ Reduce Dataset Size

Dropping rows with missing values shrinks training data — sometimes dramatically.

Why Is Data Missing in the First Place?



Sensor Failure

IoT devices and sensors drop readings during outages or interference.



Human Error

Fields left blank in forms, typos that get flagged and nulled, or skipped survey questions.



Data Merging

Joining tables with different schemas leaves gaps where records don't match up.

Understanding **why** data is missing helps you choose the right strategy to handle it.

Missing Values — It's Not So Simple

⚠️ Losing Too Much Data

If 40% of a column is missing and you drop those rows, you may lose critical patterns in your dataset.

⚠️ Introducing Bias

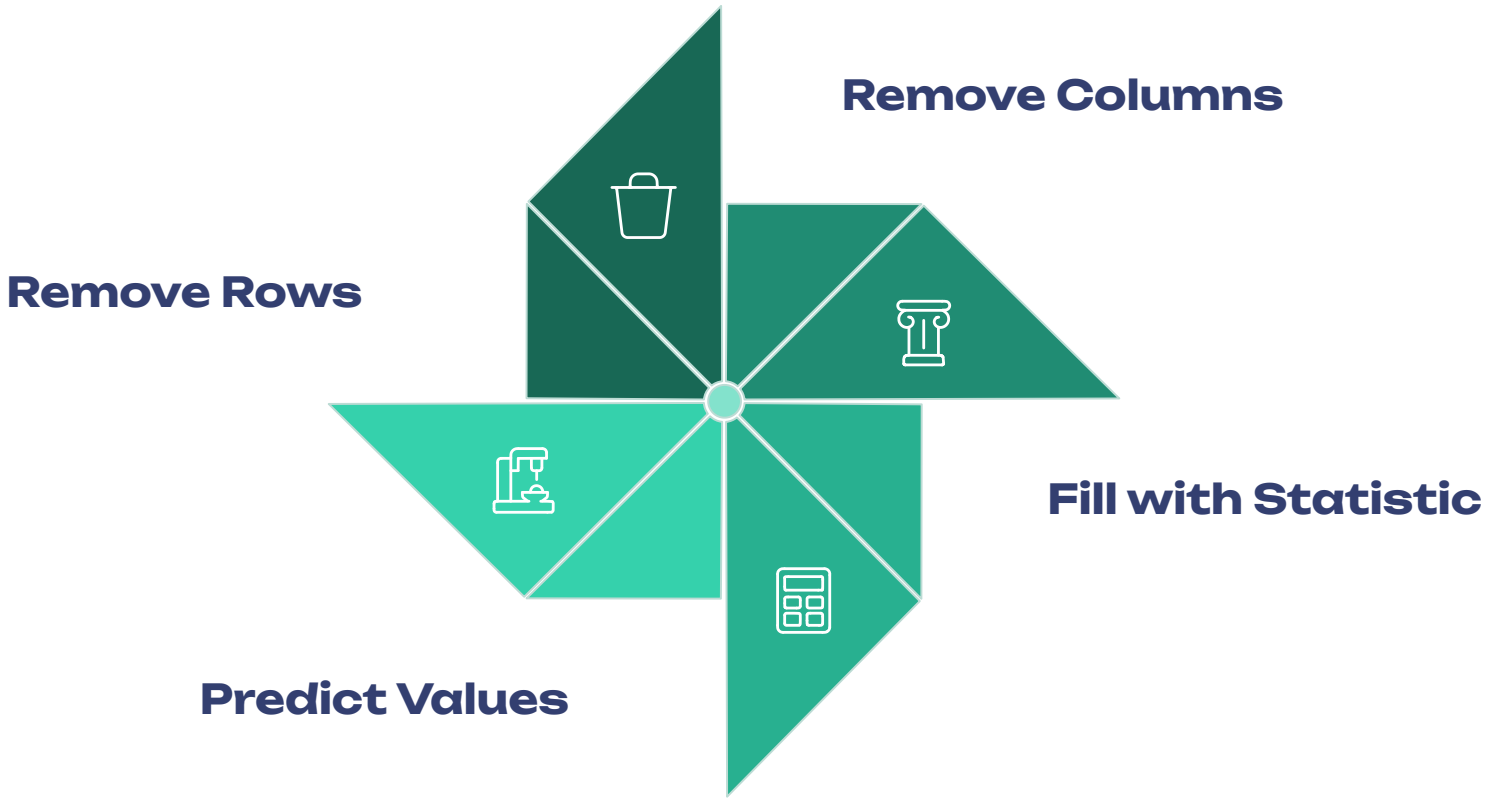
Filling every missing salary with the mean ignores that high earners may avoid disclosing income.

⚠️ No Universal "Right" Method

The best approach depends on why data is missing and how important that feature is to your model.

⚠ Every missing value strategy is a trade-off. Document your choices.

Handling Missing Values — Methods



Start simple. Only use advanced imputation (predicting missing values with a model) when you have enough data and the missing values are meaningful to preserve.

When to Remove a Column

Removing an entire column is a drastic step, but sometimes necessary when dealing with pervasive missing values.

1

High Missingness

If a column has a very high percentage of missing values (e.g., over 40%), it likely contains too little useful information to justify keeping or imputing.

2

Low Predictive Power

Even with some data, if a column with missing values shows weak or no correlation with the target variable, its presence may only add noise.

3

Unreliable Imputation

When the mechanism of missingness is complex, or there aren't enough related features to accurately estimate values, imputation can introduce more error than value.

When to Remove a Row

Deciding to remove an entire row of data is less common than removing a column, but crucial in specific scenarios. This impacts individual data points rather than entire features.

Extensive Missingness

If a row has a high percentage of missing values across many critical features, its utility for training models is minimal.

Critical Data Errors

When a row contains fundamental errors that render it factually incorrect or impossible to interpret, and correction is not feasible.

Duplicate Records

Identical rows that are true duplicates (not just similar entries) should be removed to prevent bias and inflate data volume.

⚠ Always investigate the cause of row-level issues before removal, as it directly shrinks your dataset size.



Handling Missing Values: Fill with Statistics

When dropping rows or columns isn't feasible, a common approach is to replace missing values with a calculated statistic from the existing data.

MEAN IMPUTATION

Replace missing numerical values with the **average** of the column.

- **Pros:** Simple to implement, maintains original sample size, less impact on average.
- **Cons:** Reduces variance, distorts relationships between variables, sensitive to outliers.

MEDIAN IMPUTATION

Replace missing numerical values with the **middle value** of the column.

- **Pros:** Robust to outliers, maintains sample size, better for skewed distributions.
- **Cons:** Reduces variance, can still distort relationships.

MODE IMPUTATION

Replace missing values with the **most frequent value** of the column.

- **Pros:** Applicable to categorical and numerical data, simple, maintains sample size.
- **Cons:** Can introduce bias if the mode isn't truly representative of the missing data.

The choice depends on data type, distribution, and how robust you need your model to be against imputation errors.



Data Must Be Numeric

> Machine learning models speak one language: **numbers**.

Algorithms like linear regression or neural networks compute with matrices of numbers. Text like `"Yes"`, `"Red"`, or `"High"` must be **encoded** before a model can use it.

The Problem

Categorical columns contain strings, not numbers.

The Solution

Encoding: systematically map categories to numeric representations.

Binary Encoding

When to Use It

Use binary encoding when a column has exactly **two possible values** — like Yes/No, True/False, or Male/Female.

- Simple and efficient — adds zero extra columns
- Preserves information perfectly
- Model interprets 1 vs 0 correctly

Most intuitive encoding — great starting point.

❌ Before

Name	Subscribed
Alice	Yes
Bob	No
Carol	Yes

✅ After

Name	Subscribed
Alice	1
Bob	0
Carol	1

Ordinal Encoding

✅ Before & After

Satisfaction	Encoded
Low	1
Medium	2
High	3

⚠️ Only use ordinal encoding when a **real order** exists. Don't apply it to cities or colors!

When to Use It

Use ordinal encoding when categories have a **meaningful, ranked order**.

- Low → Medium → High ✅
- Cold → Warm → Hot ✅
- Red → Blue → Green ❌

The numbers preserve the rank relationship, which models can exploit for better predictions.

One-Hot Encoding for Categories

When categories have **no natural order** — like colors, cities, or product types — we use **one-hot encoding**.

❌ Before

City
New York
London
Tokyo

✅ After — Each Category Becomes Its Own Column

City_NewYork	City_London	City_Tokyo
1	0	0
0	1	0
0	0	1

ⓘ Each row has exactly one `1` — hence "one-hot."

Encoding Challenges

Column Explosion

A City column with 500 unique values produces 500 new columns. Datasets become massive.

Memory & Speed

High-dimensional sparse data is expensive to store and slow to train on.

False Relationships

Encoding ordinal numbers into unordered data tricks models into seeing rankings that don't exist.

Alternatives for high-cardinality: target encoding, embedding layers, or frequency encoding.

Outliers — What Are They?

> "A salary of $1,000,000 in a dataset where everyone else earns $40,000–$80,000 — is that a CEO or a data entry error?"

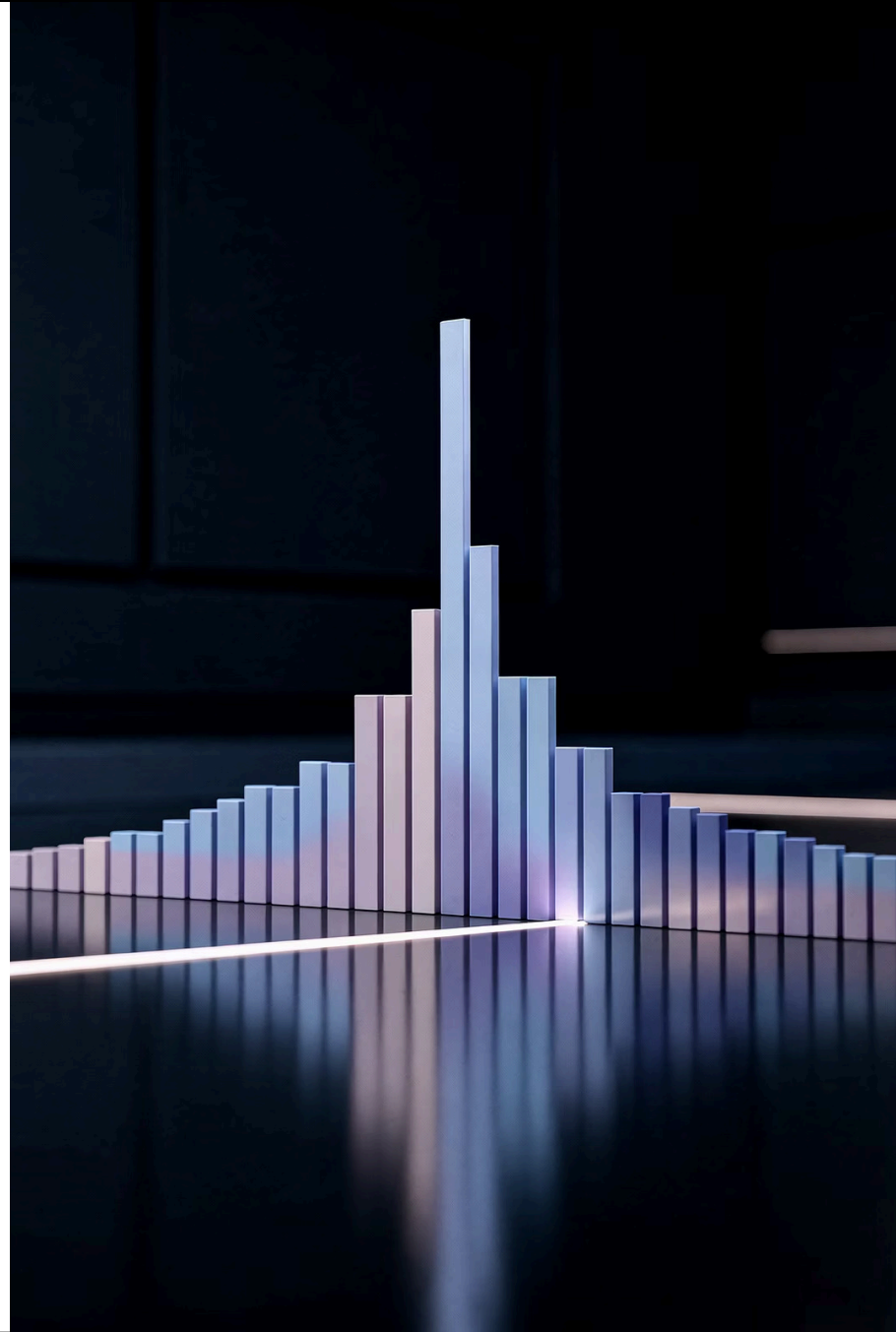
Outliers are values that lie **far from the rest of the data**. They can be genuine extreme cases — or they can be mistakes. The challenge is figuring out which.

Genuine Outlier

A real data point that happens to be extreme (e.g., a billionaire in an income survey).

Error Outlier

A typo, sensor glitch, or merge artifact — not a real observation.



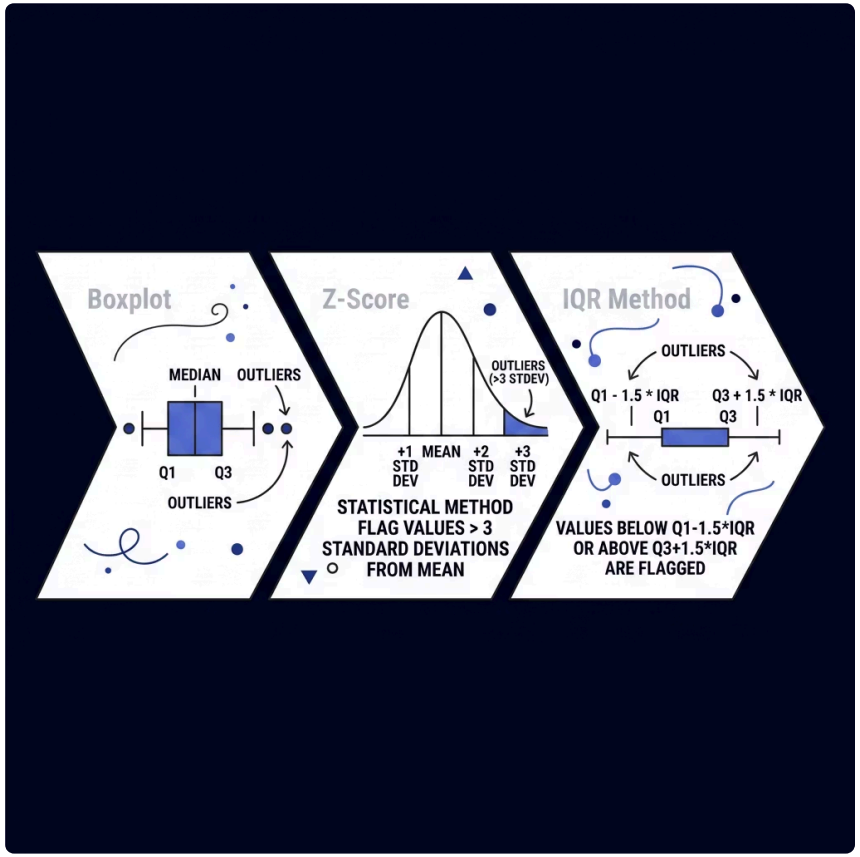
QUICK INTUITION

Detecting Outliers

Start with a boxplot — seeing the data first builds intuition before applying formulas.

Quick Intuition

- **Boxplot**: Visual — dots outside whiskers are suspects
- **Z-Score**: How many standard deviations from average?
- **IQR**: Uses the middle 50% of data as the "normal" range



Outliers — Handle With Care

🔍 Error or Reality?

Before removing anything, investigate. A house price of $5 could be a typo — or a historical sale. Context matters.

🗑️ Removing vs Keeping

Removing outliers may improve average-case accuracy but hurt performance on rare, important events (e.g., fraud detection).

📊 Impact on Models

Linear models are highly sensitive to outliers. Tree-based models (like Random Forest) are much more robust.

⚠️ Never remove an outlier just because it's inconvenient. Always have a justification.

Summary — Data Cleaning Is a Craft

01

Inspect First

Understand your data before changing it.

02

No Universal Rules

Every cleaning decision is a trade-off — document your choices.

03

Iterate Often

Cleaning is not a one-time step. Revisit as you build your model.

> 💭 Final thought: **Can two data scientists clean the same dataset differently and reach opposite model conclusions?** Discuss.





TEL-HAI
University of Kiryat Shmona
& the Galilee

Thank You!

We appreciate your time and interest in understanding the nuances of data cleaning. We look forward to collaborating on your data science initiatives.

Email Us

MarcelFei@telhai.ac.il

Our Office

Building 6, Room 309, Eastern Campus,
Tel Hai Kyriat Shmona

Reception Hours

By Appointment Only